



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Let's get the hang of variables in C#!



Learning Outcome:

Talk about principles, concepts, characteristics of general-purpose computer programming language in spoken and written situations.

Ask your partner:

- Do you know what the function of storage areas in programming languages is?
- What is an initializer in programming languages?

Can Do Statements:

- I can talk about the concept of variables in C#.
- I can identify the syntax for variable definition in C#.
- I can discuss the syntax for initializing variables in C#.





Let's Explore!

Talking about variables in C#



Instructions: Read the text below.

- Manager:** Baxter! You'd better explain to me why everybody except you is attending the compulsory training sessions this week.
- Baxter:** Sorry. It's just that I learned all about concept variables in C# in my last job. In fact, I deliver a training session about it next week.
- Manager:** Oh, really? Well, if you're so smart, you had better tell me what variables in C# are exactly?
- Baxter:** Well, variables in this programming language are storage areas that our programs can manipulate. They have a specific type which determines how they will store data.
- Manager:** Okay, Baxter. What about the syntax?
- Baxter:** Let me write this on the board: `< data_type >` must be a valid C# data type such as `char`, `int`, `float`, `double`, or any user-defined data type. Look: `< variable_list >` may consist of one or more identifier names separated by commas. Some valid variable definitions are `"int i, j, k; char c, ch; float f, salary; double d;"`
- Manager:** Baxter, you weren't lying. What about the syntax for initializing variables?
- Baxter:** Like I said, I'm actually an expert in C#. You initialize a variable using an equal sign followed by a constant expression. The general form of initialization is: `variable_name = value;`. Variables can be initialized in their declaration. The initializer consists of an equal sign followed by a constant expression as: `< data_type > < variable_name > = value;`
- Manager:** Seriously, I'm lost for words; you really know your stuff.
- Baxter:** Thanks. The engineers are coming this week, right? If they need a hand with anything, I am available after 3:00 p.m. every day.
- Manager:** You ought to drop by my office this week and we can see if there are any positions in the programming department more suitable for your talents.
- Baxter:** That'd be great.



Let's Practice!

Some key movements when initializing variables in C#

Instructions: Use the words from the box to correctly complete the sentences. Be careful with the distractors.

Word Bank

ought to / had better / equal sign / have to be better / constant expression
/ sign of equal / you'd better / should to.

1. You _____ know how the C# variables work.
2. Since you're an expert on C#, you _____ get promoted.
3. For initializing variables in C#, you'd better write the _____.
4. You ought to write the equal sign and the _____.
5. _____ put the equal sign before the constant expression.



Let's Practice!

Are they talking about C# variables in future?

Instructions: Select the correct category in the table. There is only one answer per category.

	Present Continuous for Future Events	Simple Present for Future Events
The engineers are coming this week, right?		
I deliver a training session about it next week.		
I'm explaining that general form of initialization this afternoon in the training session.		
He is explaining that concept tomorrow in class.		
The manager is promoting the engineer because he knows how to handle variables in C#.		



Extension Activity

Learning More About Variables in C#

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for more information about variables in C#.
- Describe every type of value in detail.
- Make a poster with those descriptions.
- Share your poster with the rest of the class.

Self-Assessment

Yes

No

- I look for more information about variables in C#.
- I describe every type of value in detail.
- I make a poster with the descriptions.
- I share my poster with the rest of the class.

☐ ☐
☐ ☐
☐ ☐
☐ ☐


Target Words

- **need a hand** (expression): to need help
e.g. If they need a hand with anything, I am available after 3:00pm every day.
- **drop by** (phrasal verb): to come
e.g. You ought to drop by my office this week.
- **department** (noun): an area of special expertise in an organization
e.g. We can see if there are any positions in the programming department more suitable for your talents.
- **compulsory** (adjective): obligatory
e.g. You'd better explain to me why everybody except you is attending the compulsory training sessions this week.
- **deliver** (verb): to give
e.g. I deliver a training session about it next week.



Learning Points

Present Continuous and Simple Present to Talk About Upcoming Events

Present continuous tense and simple present tense are commonly used to talk about events or situations that take place in the present. However, we can also use these tenses to describe future events.

The **present continuous** can have a future meaning. This is used for future arrangements.

Look at the following structure:

Subject + **verb to be (am/is/are)** + **verb in gerund (-ing)** + complement

Example:

We **are reading** about variables next Monday.

The **present simple** can also have a future meaning. We use it in two situations:

- Timetabled events
- Sentence with future meaning

In order to write future statements, we follow the usual structure of the present simple. However, we have to specify the time when the action will happen.

Examples:

The class on variables **starts** at 11:00.

We **begin** with variables after we finish this topic.

Examples

- *The engineers are coming this week, right?*
- *In fact, I deliver a training session about it next week..*

Basic Value Types Provided in C#

There are different categories of **value types in C#**. Take a look at the following chart in which there are examples of each type.

Type	Example
Integral types	Sbyte, byte, short, ushort, int, uint, long, ulong
Floating point types	Float and double
Decimal types	Decimal
Boolean types	True or false values, as assigned
Nullable types	Nullable data types

